

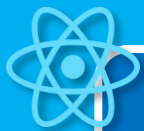


FRONT-END DEVELOPER



Bài 5 – Lifecycle và Side Effect

Môn học: Lập trình Front-end



Nội dung



1. Side Effects



2. Phân loại useEffect

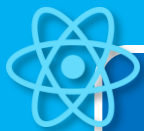


3. Các lỗi thường gặp khi sử dụng useEffect



4. Tương tác API



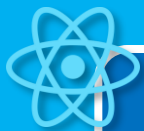


Side Effects

- ❑ Trong lập trình hàm (functional programming), một function lý tưởng chỉ nên nhận đầu vào (props/state) và trả về đầu ra (UI). Khi một function thực hiện một hành động ảnh hưởng đến các yếu tố bên ngoài phạm vi của nó, đó gọi là **Side Effect**.
- ❑ Các Side Effects phổ biến trong React:
 - **Gọi API (Data Fetching):** Lấy dữ liệu từ server.
 - **Tương tác DOM:** Thay đổi title của document, tính toán kích thước phần tử.
 - **Đăng ký (Subscriptions):** Lắng nghe sự kiện (event listeners), kết nối WebSocket.
 - **Timers:** Sử dụng setTimeout hoặc setInterval.

React yêu cầu quá trình render phải "tinh khiết"
(pure) → useEffect

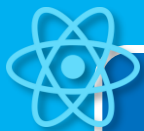
7/31/2026



Cấu trúc của useEffect

```
useEffect(() => {  
  // 1. Setup code (Thực thi Side Effect)  
  return () => {  
    // 3. Cleanup code (Dọn dẹp Side Effect - Tùy chọn)  
  };  
}, [/* 2. Dependency Array (Mảng phụ thuộc - Tùy chọn) */]);
```

- ❑ **Setup Code:** Nơi chứa logic Side Effect của bạn. Nó luôn chạy ít nhất một lần sau khi component được mount và trình duyệt đã vẽ (paint) UI xong.
- ❑ **Dependency Array:** Mảng chứa các biến (state, props). React sẽ so sánh giá trị của chúng giữa các lần render để quyết định xem có cần chạy lại Effect hay không.
- ❑ **Cleanup Code:** Một hàm được return để "dọn dẹp" những gì Setup code đã tạo ra, giúp tránh rò rỉ bộ nhớ (memory leaks).



Quản lý vòng đời (Lifecycle) qua Dependency Array

Trước khi React Hooks (như `useEffect`) ra đời, các lập trình viên xây dựng giao diện bằng **Class Component**. Mỗi component này trải qua 3 giai đoạn chính trong vòng đời của nó: **Mounting** (Sinh ra / Xuất hiện), **Updating** (Cập nhật), và **Unmounting** (Hủy bỏ / Biến mất).

1. `componentDidMount()` — Giai đoạn "Sinh ra" (Mounting)

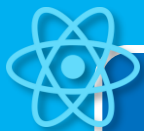
Phương thức này được React gọi **một lần duy nhất** ngay sau khi component lần đầu tiên được render và chèn (mounted) thành công vào cây DOM thực của trình duyệt.

Thời điểm chạy: Chạy ngay sau lần render đầu tiên.

Mục đích chính: Vì lúc này phần tử HTML đã thực sự tồn tại trên trình duyệt, đây là nơi an toàn và lý tưởng nhất để thực hiện các Side Effects cần tương tác với bên ngoài.

Ứng dụng thực tế:

- ☐ Gọi API lấy dữ liệu từ server để nạp vào giao diện.
- ☐ Gắn các sự kiện (event listeners) toàn cục (ví dụ: lắng nghe sự kiện cuộn chuột `window.addEventListener`).
- ☐ Tương tác trực tiếp với DOM (ví dụ: khởi tạo các thư viện đồ thị, tính toán chiều cao của một thẻ `<div>`).
- ☐ Thiết lập các bộ đếm thời gian (`setInterval`, `setTimeout`).



Quản lý vòng đời (Lifecycle) qua Dependency Array

2. `componentDidUpdate(prevProps, prevState)` — Giai đoạn "Cập nhật" (Updating)

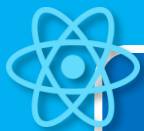
Giai đoạn này xảy ra mỗi khi component nhận được dữ liệu mới (Props thay đổi) hoặc trạng thái nội bộ của nó thay đổi (State thay đổi), khiến nó phải render lại. Phương thức này chạy **ngay sau khi** quá trình render lại hoàn tất và DOM đã được cập nhật.

Thời điểm chạy: Chạy sau mỗi lần component re-render (ngoại trừ lần render đầu tiên).

Tham số nhận vào:

- ❑ `prevProps`: Giá trị của props ở lần render trước đó.
- ❑ `prevState`: Giá trị của state ở lần render trước đó.

Mục đích chính: Dùng để thực hiện các hành động dựa trên sự **thay đổi** của dữ liệu. Bằng cách so sánh dữ liệu cũ (`prev`) và dữ liệu mới (`this.props` hoặc `this.state`), bạn có thể quyết định xem có cần chạy logic nào đó hay không.



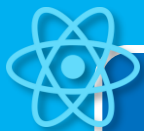
Quản lý vòng đời (Lifecycle) qua Dependency Array

Quy tắc: Nếu bạn gọi hàm cập nhật state (`this.setState`) bên trong phương thức này, bạn **bắt buộc** phải bọc nó trong một câu lệnh `if` để kiểm tra điều kiện. Nếu không, nó sẽ tạo ra một vòng lặp vô hạn (Infinite Loop: `update -> re-render -> update -> re-render...`).

// Ví dụ về quy tắc sinh tử trong `componentDidUpdate`:

```
componentDidUpdate(prevProps) {  
  // Chỉ gọi API lấy thông tin user MỖI nếu ID của user bị thay đổi  
  if (this.props.userId !== prevProps.userId) {  
    this.fetchUserData(this.props.userId);  
  }  
}
```

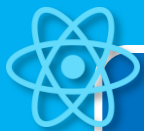
Trong Functional Component, nó tương đương với việc sử dụng `useEffect` có chứa các biến trong mảng phụ thuộc [`userId`].



Quản lý vòng đời (Lifecycle) qua Dependency Array

3. `componentWillUnmount()` — Giai đoạn "Hủy bỏ" (Unmounting)

- ❑ **Thời điểm chạy:** Chạy ngay trước khi component bị hủy (destroy) và gỡ khỏi DOM.
- ❑ **Mục đích chính:** Thực hiện công tác "dọn dẹp" (cleanup) để giải phóng tài nguyên, tránh gây rò rỉ bộ nhớ (memory leaks).
- ❑ **Ứng dụng thực tế:**
 - Xóa bỏ các bộ đếm thời gian (`clearInterval`, `clearTimeout`) đã tạo ở `componentDidMount`.
 - Gỡ bỏ các event listeners (`window.removeEventListener`).
 - Hủy bỏ các kết nối mạng đang dang dở (ví dụ: abort API requests) hoặc ngắt kết nối WebSocket.



Quản lý vòng đời (Lifecycle) qua Dependency Array

useEffect gộp cả 3 phương thức vòng đời (componentDidMount, componentDidUpdate, componentWillUnmount) vào một API duy nhất thông qua cách bạn khai báo **Dependency Array**.

Trường hợp 1: Không có mảng phụ thuộc (Chạy liên tục)

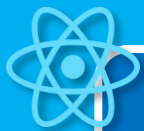
Effect sẽ chạy sau **mỗi lần component render**. Điều này rất dễ gây ra vòng lặp vô hạn (infinite loop) nếu bên trong Effect bạn cập nhật state.

```
useEffect(() => {  
  console.log("Chạy sau mỗi lần render!");  
});
```

Trường hợp 2: Mảng phụ thuộc rỗng [] (Tương đương componentDidMount)

Effect chỉ chạy **duy nhất một lần** sau lần render đầu tiên. Thường dùng để gọi API khởi tạo hoặc gán các event listener toàn cục.

```
useEffect(() => {  
  console.log("Chỉ chạy 1 lần duy nhất khi component xuất hiện (Mount)!");  
}, []);
```

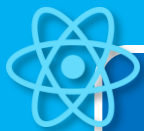


Quản lý vòng đời (Lifecycle) qua Dependency Array

Trường hợp 3: Mảng phụ thuộc có giá trị (Tương đương `componentDidUpdate`)

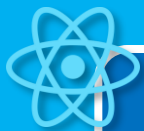
Effect sẽ chạy sau lần render đầu tiên **VÀ** chạy lại bất cứ khi nào một trong các giá trị trong mảng bị thay đổi giữa các lần render.

```
useEffect(() => {  
  console.log(`Tôi chạy khi userId thay đổi. UserId hiện tại là: ${userId}`);  
  // Lấy dữ liệu user mới mỗi khi userId thay đổi  
}, [userId]);
```



So sánh

Giai đoạn	Tên phương thức	Tương đương trong useEffect	Tác vụ chính
Mount	componentDidMount	useEffect(..., [])	Gọi API, gắn event, khởi tạo.
Update	componentDidUpdate	useEffect(..., [deps])	So sánh props/state cũ và mới để xử lý logic.
Unmount	componentWillUnmount	return () => {} trong Effect	Dọn dẹp tài nguyên (clear timer, remove event).



Race Condition

(Xung đột tiến trình / Xung đột dữ liệu)

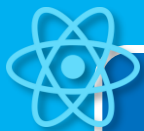
❑ Race Condition xảy ra khi **thứ tự hoàn thành của các tác vụ bất đồng bộ (như gọi API) không tuân theo thứ tự mà chúng được khởi chạy**. Dữ liệu hiển thị trên giao diện phụ thuộc vào việc "ai về đích cuối cùng" thay vì "ai được gọi cuối cùng".

❑ **Ngữ cảnh xảy ra:**

Người dùng thao tác nhanh làm gửi nhiều yêu cầu API liên tiếp (ví dụ: chuyển từ trang User ID=1 sang ID=2 rồi sang ID=3).

[Thời điểm 1] Gọi API User 1 -----> [Xử lý mất 3s] ---> Kết quả 1 về (GHI ĐỀ!)

[Thời điểm 2] Gọi API User 2 ----> [Xử lý 1s] -> Kết quả 2 về

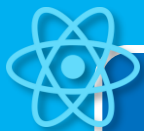


Race Condition

(Xung đột tiến trình / Xung đột dữ liệu)

- ❑ Sử dụng **Cleanup function** trong `useEffect` với biến cờ `ignore` hoặc ngắt kết nối bằng `AbortController`.

```
useEffect(() => {  
  let isCurrent = true;  
  async function fetchData() {  
    const res = await fetch(`/api/user/${userId}`);  
    const data = await res.json();  
    if (isCurrent) setUser(data);  
    // Chỉ cập nhật nếu request này vẫn là mới nhất  
  }  
  fetchData();  
  return () => {  
    isCurrent = false;  
    // Vô hiệu hóa kết quả của request cũ khi userId thay đổi  
  };  
}, [userId]);
```



Memory Leak (Rò rỉ bộ nhớ)

Memory Leak xảy ra khi một chương trình **chiếm dụng dung lượng bộ nhớ RAM nhưng không giải phóng nó** khi không còn cần dùng đến nữa.

Ngũ cảnh xảy ra trong React:

- ☐ Component đã bị dọn dẹp khỏi DOM (**Unmounted** — ví dụ: chuyển sang trang khác), nhưng các tác vụ ngầm của nó vẫn tiếp tục chạy trong bộ nhớ:
- ☐ Bộ đếm thời gian (setInterval, setTimeout) vẫn chạy ngầm.
- ☐ Lắng nghe sự kiện toàn cục (window.addEventListener) vẫn tiếp tục nhận event.
- ☐ API trả về kết quả và code cố gắng thực hiện setState() trên một component "đã chết".



Memory Leak (Rò rỉ bộ nhớ)

Hậu quả:

- ❑ Trình duyệt ngốn ngày càng nhiều RAM.
- ❑ Ứng dụng chậm dần, giật lag hoặc bị crash trên thiết bị cấu hình yếu.
- ❑ Xuất hiện cảnh báo trên Console (trong các phiên bản React cũ):
"Can't perform a React state update on an unmounted component..."

Cách khắc phục:

- ❑ Luôn **dọn dẹp (Cleanup)** tất cả các tác vụ đăng ký ngầm trong hàm return của useEffect.

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Đang chạy ngầm...");  
  }, 1000);  
  // Dọn dẹp bộ đếm khi component unmount  
  return () => clearInterval(timer);  
}, []);
```



Infinite Loop (Vòng lặp vô hạn)

Infinite Loop xảy ra khi quá trình **Render -> Chạy Effect -> Cập nhật State -> Re-render** bị lặp đi lặp lại không có điểm dừng.

Ngữ cảnh xảy ra:

- ☐ Quên khai báo **Dependency Array** trong `useEffect` khi có thao tác cập nhật state.
- ☐ Đặt một **Object/Array** trực tiếp vào `Dependency Array` mà không kiểm soát việc khởi tạo lại tham chiếu (reference).

// **✗ Nguyên nhân phổ biến: Quên mảng phụ thuộc []**

```
useEffect(() => {  
  fetchData().then(data => {  
    setData(data); // 1. Cập nhật State  
  });  
});
```

// **2. Thiếu [], Effect chạy sau MỖI LẦN RENDER -> Tạo vòng lặp vô hạn!**



Infinite Loop (Vòng lặp vô hạn)

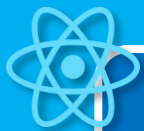
Component Render ban đầu → useEffect chạy → setData() làm State thay đổi → Component Re-render → useEffect lại chạy → setData() ... (Lặp lại vĩnh viễn)

Hậu quả:

- ☐ Trình duyệt bị treo đứng lập tức (Freeze).
- ☐ Gửi hàng nghìn request liên tục tới Server trong vài giây -> Làm **sập API/Server** (vô tình tấn công DDoS server của chính mình).

Cách khắc phục:

- ☐ **Luôn khai báo mảng phụ thuộc** cho useEffect. Nếu chỉ muốn chạy 1 lần khi mount, hãy truyền mảng rỗng [].
- ☐ Tránh đưa các giá trị dạng object/array được khởi tạo lại ở mỗi lần render vào mảng phụ thuộc (hoặc bọc chúng bằng useMemo / useCallback).



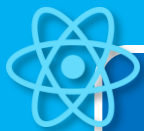
Vai trò của Dependency Array khi gọi API

Mảng phụ thuộc (Dependency Array) quyết định **khi nào** yêu cầu API được gửi đi.

Gọi API 1 lần duy nhất khi Mount ([])

- ❑ Dùng cho các dữ liệu tĩnh hoặc dữ liệu khởi tạo của trang (danh sách sản phẩm ban đầu, thông tin user hiện tại).

```
useEffect(() => {  
  const fetchProducts = async () => {  
    const response = await fetch('https://api.example.com/products');  
    const data = await response.json();  
    setProducts(data);  
  };  
  fetchProducts();  
}, []); // Mảng rỗng -> Chỉ gọi 1 lần sau lần render đầu tiên
```



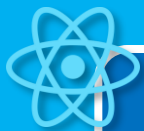
Vai trò của Dependency Array khi gọi API

Gọi lại API khi tham số thay đổi ([dep1, dep2])

- ❑ Dùng cho các tính năng như: Tìm kiếm (search), Phân trang (pagination), Lọc sản phẩm (filter), hoặc Xem chi tiết theo ID.

```
useEffect(() => {  
  const fetchUserDetail = async () => {  
    setLoading(true);  
    const response = await fetch(`https://api.example.com/users/${userId}`);  
    const data = await response.json();  
    setUser(data);  
    setLoading(false);  
  };  
  fetchUserDetail();  
}, [userId]);
```

// Mỗi khi userId thay đổi, Effect sẽ chạy lại để lấy dữ liệu user mới



Vai trò của Dependency Array khi gọi API

Cảnh báo lỗi Infinite Loop (Vòng lặp vô hạn)

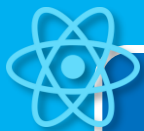
// **✗ LỖI NGHIÊM TRỌNG:** Vòng lặp vô hạn!

```
useEffect(() => {  
  fetch('/api/data')  
    .then(res => res.json())  
    .then(data => setData(data));
```

// Thay đổi State->Re-render->Chạy lại Effect -> Gọi lại API...

```
}); // THIẾU MẢNG PHỤ THUỘC!
```

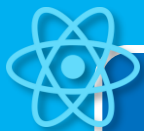
Quy tắc: Luôn luôn cung cấp mảng phụ thuộc [] hoặc các biến cụ thể khi gọi API trong useEffect.



Vai trò của Cleanup Function khi tương tác với API

Trong môi trường thực tế, mạng (network) không phải lúc nào cũng phản hồi ngay lập tức. Nếu người dùng thao tác nhanh (ví dụ: chuyển trang liên tục hoặc chuyển qua lại giữa các tab người dùng `userId = 1` sang `userId = 2`), hai vấn đề nguy hiểm sau sẽ xảy ra:

- 1. Race Condition (Xung đột tiến trình):** API của `userId = 1` phản hồi chậm hơn API của `userId = 2`. Dù người dùng đang ở `userId = 2`, dữ liệu hiển thị cuối cùng lại là của `userId = 1` (do phản hồi sau).
- 2. Memory Leak / State update on unmounted component:** Component đã bị đóng/xóa khỏi màn hình nhưng API phản hồi về và vẫn cố gọi `setState()`.



Vai trò của Cleanup Function khi tương tác với API

Giải pháp 1: Dùng biến cờ hiệu ignore (Đơn giản & Khuyến dùng)

- ❑ Biến cờ này giúp bỏ qua kết quả API cũ nếu Effect đã được dọn dẹp (cleanup) trước khi API phản hồi xong.

```
useEffect(() => {
```

```
  let isCurrent = true;
```

```
  // Biến cờ đánh dấu Effect hiện tại còn hiệu lực không
```

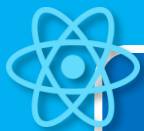
```
  async function fetchPostData() {
```

```
    setLoading(true);
```

```
    const res = await
```

```
    fetch(`https://jsonplaceholder.typicode.com/posts/${postId}`);
```

```
    const data = await res.json();
```



Vai trò của Cleanup Function khi tương tác với API

// Chỉ cập nhật State nếu Effect này vẫn đang là Effect mới nhất

```
if (isCurrent) {  
  setPost(data);  
  setLoading(false);  
}
```

```
}
```

```
fetchPostData();
```

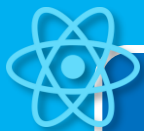
// Cleanup Function: Chạy khi postId thay đổi HOẶC khi Component bị Unmount

```
return () => {  
  isCurrent = false;
```

// Đánh dấu dữ liệu của yêu cầu này không còn cần thiết nữa

```
};
```

```
}, [postId]);
```



Vai trò của Cleanup Function khi tương tác với API

Giải pháp 2: Hủy trực tiếp yêu cầu mạng với AbortController (Chuẩn Web API)

Thay vì chờ API phản hồi rồi bỏ qua kết quả như cách 1, AbortController sẽ **ngắt hẳn kết nối mạng** ngay khi người dùng chuyển sang thao tác mới.

```
useEffect(() => {
```

```
  // 1. Tạo một controller quản lý ngắt tín hiệu
```

```
  const controller = new AbortController();
```

```
  const { signal } = controller;
```

```
  const fetchData = async () => {
```

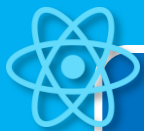
```
    try {setLoading(true);
```

```
      const response = await
```

```
      fetch(`https://api.example.com/search?q=${searchTerm}`, { signal });
```

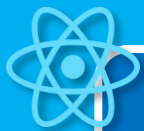
```
      const data = await response.json();
```

```
      setResults(data);
```

Vai trò của Cleanup Function khi tương tác với API

```
} catch (error) {  
  if (error.name !== 'AbortError') {  
    // Chỉ xử lý lỗi nếu đó KHÔNG PHẢI lỗi do ta chủ động hủy request  
    setError(error.message);  
  }  
} finally {  
  setLoading(false);  
}  
};  
if (searchTerm) { fetchData();}  
// 2. Cleanup Function: Hủy yêu cầu mạng cũ nếu searchTerm thay đổi  
return () => { controller.abort(); };  
}, [searchTerm]);
```

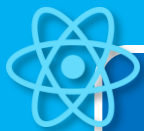


Quy trình chuẩn cho một Component gọi API

Ví dụ hoàn chỉnh kết hợp **Loading State**, **Error State**, **Dependency Array** và **Cleanup Function** (sử dụng AbortController):

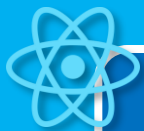
```
import React, { useState, useEffect } from 'react';
export default function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  useEffect(() => {
    const controller = new AbortController();
    async function loadUserData() {
      try {
        setLoading(true);
        setError(null);

```



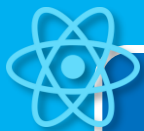
Quy trình chuẩn cho một Component gọi API

```
const response = await
fetch(`https://jsonplaceholder.typicode.com/users/${userId}`, {
  signal: controller.signal
});
if (!response.ok) {
  throw new Error('Không thể lấy dữ liệu từ server');
}
const data = await response.json();
setUser(data);
} catch (err) {
  if (err.name !== 'AbortError') {
    setError(err.message);
  }
}
```



Quy trình chuẩn cho một Component gọi API

```
finally {setLoading(false); }  
  loadUserData();  
  // Cleanup: Ngắt request nếu userId đổi trước khi API trả về kết quả  
  return () => controller.abort();  
}, [userId]);  
if (loading) return <p>Đang tải dữ liệu người dùng...</p>;  
if (error) return <p style={{ color: 'red' }}>Lỗi: {error}</p>;  
if (!user) return null;  
return (  
  <div><h3>{user.name}</h3><p>Email: {user.email}</p>  
    <p>Phone: {user.phone}</p></div>  
);  
}
```



Quy tắc khi dùng useEffect tương tác với API

- ❑ **Khai báo hàm async bên trong Effect:** Không được biến callback chính của useEffect thành async (useEffect(async () => ...) là sai cú pháp). Hãy tạo một hàm async nhỏ bên trong rồi gọi nó.
- ❑ **Khai báo đủ Dependencies:** Bất kỳ tham số nào lấy từ props hoặc state dùng trong chuỗi URL gọi API (như userId, searchTerm, page) **bắt buộc** phải khai báo vào Dependency Array.
- ❑ **Thực hiện Cleanup:** Luôn sử dụng biến cờ ignore hoặc AbortController khi API phụ thuộc vào các tham số động để chống **Race Condition**.
- ❑ **Quản lý trạng thái đầy đủ:** Đảm bảo luôn xử lý đủ 3 trạng thái của một yêu cầu API: Data (Thành công), Loading (Đang xử lý) và Error (Thất bại).